REQUI TRADING · DEVELOPMENT GUIDE

Intelligence & Self-Diagnosis Upgrade

Conversation memory · Learning loop · GPT-5.6 · System Check module with AI diagnostics

Audience: DevOps / backend engineer integrating this add-on

Scope: what changed vs. the previous version, file placement, integration steps, full source

Platform: React 19 + tRPC + Drizzle + MySQL · Hono server · autonomous strategy engine

Date: August 2026

Contents

1. What is new in this version

This version adds four capabilities on top of the autonomous engine and the Intelligence tool-use runtime you already have. Everything funnels through the same constitutional confirmation gate — nothing in this package widens execution authority.

1.1 Conversation memory

The Intelligence chat was stateless: every message arrived with zero context. A new `chat_messages` table now persists each exchange, and every chat request loads the last 20 messages into the model. The assistant remembers names, open threads, and earlier decisions. Retention is bounded (400 rows per user, auto-pruned), failures degrade gracefully (a memory outage never breaks a chat), and the user can wipe their own history via a new endpoint.

1.2 Learning loop — the "machine learning" component

This is a deterministic learning layer, not neural-network training — the honest mechanism for a trading system. A new `trade_outcomes` table records every completed trade with its full decision context (strategy, variant, entry/exit, R-multiple, exit reason, auto-classified lesson). A diagnosis engine aggregates per-variant performance, classifies mistake patterns (stop-outs vs. stalls vs. fades), flags when live results diverge from backtest expectancy, and distills lessons. Those lessons are injected into every Intelligence conversation, so the AI advises from *our* logged outcomes — and a new `getLearningReport` tool lets the AI answer "what keeps failing?" from real data. Every backtest (UI or AI-initiated) now feeds the loop automatically.

1.3 Model upgrade to GPT-5.6

The default model moves from `gpt-4.1-mini` to `gpt-5.6` (OpenAI's current frontier generation, July 2026). It remains one env-var override (`OPENAI_MODEL`) — no code change needed to move between the Sol / Terra / Luna tiers.

1.4 System Check module with AI diagnostics

A self-check subsystem — the platform's immune system. Eleven real component probes (environment, database, governance, market data feed, engine configs, a signal-stack self-test that proves the trigger math fires, autonomous monitors, the ticket gate, a live LLM ping, memory, learning) run automatically **before market open (09:00 ET)** and **after close (16:15 ET)**, Mon–Fri, plus on demand. When anything is degraded, GPT-5.6 produces a structured diagnosis — what is broken, likely cause, exact fix steps, prevention, and growth suggestions. Reports persist to a new `system_check_reports` table, results push to the owner's in-app notification center, and a new owner console page shows live status, history, and the AI diagnosis.

> **Boundary, stated plainly:** the System Check module observes and advises. Its only automatic action is restarting the market-data refresh loop if it is down during market hours. It never changes config, never loosens a clamp, never touches governance, and never executes anything.

2. Version comparison — before vs. now

Area	Previous version	This version
Chat context	Stateless — system prompt + live snapshot + single user message; no continuity between messages.	Persistent per-user history (`chat_messages`); last 20 messages injected per chat; user can view/clear via `intelligence.history` / `intelligence.clearHistory` .
Learning	None. Backtests returned numbers and were forgotten; the AI gave generic advice with no knowledge of what had failed before.	`trade_outcomes` log + aggregation engine: per-variant stats, mistake classification, live-vs-backtest divergence flags, distilled lessons injected into the snapshot; new `getLearningReport` tool; new `engine.learning` endpoint.
Model	`gpt-4.1-mini` default.	`gpt-5.6` default (env override unchanged).
System health	No automated checks. Failures surfaced only when a user hit them.	11-probe System Check, scheduled pre-open/post-close ET + manual; AI diagnosis; owner page; in-app notifications.
Intelligence tools	8 whitelisted tools.	9 tools — adds `getLearningReport` .
Snapshot	Data feed, monitors, tickets, alerts, governance, configs.	Same + `LEARNING` section (distilled lessons from logged outcomes).
Runtime rules	6 addendum rules.	7 — adds memory & learning usage rule.
DB tables	12 tables (incl. governance).	15 — adds `chat_messages` , `trade_outcomes` , `system_check_reports` . All created idempotently at boot.
Routers	`marketData` , `engine` , …	+ `systemCheck` (admin-only); `engine` gains `learning` ; `intelligence` gains `history` / `clearHistory` ; `engine.backtest` records outcomes.
Owner console	Overview, Users, Billing, Connectors, Governance, Support.	+ **System Check** page (live probes, AI diagnosis, run history, scheduler status).
Boot	ensureSchema + governance genesis.	+ `ensureSystemCheckLoop()` scheduler start.

> **Backward compatibility:** purely additive. No existing table is altered, no route signature changed, no prompt text removed. A deployment that never sets `OPENAI_API_KEY` still boots — Intelligence degrades to fallback answers and System Check reports mark the LLM probe as a warning, not a failure.

3. Architecture — how Intelligence talks to the other systems

The Intelligence layer is not a chatbot bolted on the side — it is wired into the engine, the data feed, the ticket gate, and now memory and learning, through one server-enforced tool whitelist. Everything it can do is a function the server exposes; everything it says can be grounded in live system state.

3.1 The four connections

Connection	Mechanism	What it makes possible
Intelligence ↔ Engine	Whitelisted tools wrap the deterministic engine directly: `runBacktest`, `simulateVariants`, `scanWatchlist`, `getIndicators`, `getMonitors`.	The AI answers from real triggers, timers, and variant ladders — and explains engine evidence in plain language instead of speaking generically.
Intelligence ↔ Memory	`loadHistory()` injects the last 20 messages after the system prompt; `saveMessage()` persists each exchange; bounded retention with pruning.	Continuity: the AI remembers prior decisions, open threads, and user preferences across the conversation.
Intelligence ↔ Learning	`learningSummaryLines()` appends a LEARNING section to the live snapshot; the `getLearningReport` tool exposes the full diagnosis (variant stats, mistake patterns, divergence flags).	The AI's advice improves as outcomes accumulate — it deprioritizes weak variants, respects divergence flags, and answers "what keeps failing" from logged fact.
Intelligence ↔ Execution	The single write tool, `proposeTrade`, calls the existing `proposeTicket()` — the same constitutional gate everything else uses. Stop price is required at code level.	Natural-language trade intent becomes a staged ticket. Nothing reaches a broker without the exact `CONFIRM ORDER [TICKET_ID]` from a human.

3.2 The learning feedback loop

The learning loop is a closed circuit with three stages, implemented in `api/engine/learning.ts`:

1. **Record** — every backtest (from the Engine panel, the tRPC endpoint, or the AI's own `runBacktest` tool) writes each trade to `trade_outcomes` with strategy, variant, prices, R-multiple, exit reason, and an auto-classified lesson ("stopped out — entry quality or stop placement", "stalled — no follow-through", "faded into the close", …). Paper and live outcomes hook into the same table as those paths close trades.
2. **Diagnose** — `getLearningReport()` aggregates per strategy+variant+source: win rate, expectancy (R), profit factor. It classifies losing trades into mistake categories, identifies the weakest variant with a meaningful sample, and raises a **divergence flag** when live expectancy deviates from backtest expectancy by more than 0.5R over 5+ live trades — the build spec's "if live results diverge from backtest, flag it" rule, implemented.
3. **Feed back** — distilled lessons enter the Intelligence snapshot (so the AI advises from our outcomes) and the report is available to the AI (`getLearningReport` tool), to the UI (`engine.learning`), and to the System Check module (learning-table probe).

> **What learning never touches:** sizing math, governance clamps, the confirmation gate. Learning adjusts *ranking and*

> *advice*. Authority stays exactly where the constitution put it.

3.3 The System Check immune system

System Check inverts the direction of intelligence: instead of the AI answering questions about the system, the system interrogates itself and the AI explains the results.

- **Probes** (`checks.ts`) are executable assertions, not pings. The signal self-test feeds synthetic bars through the real `vwap_reclaim` trigger and asserts it arms on a textbook pattern and stays quiet on a no-dip session; risk sizing is asserted against the $500 / 25% ceilings. A regression in trigger math fails the check before it can produce a bad proposal.
- **Scheduler** (`service.ts`) fires pre-open (09:00 ET) and post-close (16:15 ET), Mon–Fri, latched per day per window, plus manual runs from the owner console.
- **Diagnosis** — on any WARN/FAIL, the failures go to GPT-5.6 with a reliability-engineer prompt that is constitutionally constrained (it is instructed never to suggest loosening risk limits or bypassing governance) and returns BROKEN / PREVENT / GROWTH sections. The post-close run requests growth suggestions even when all checks pass.
- **Reporting** — every run persists to `system_check_reports` ; admins receive in-app notifications through the existing alerts table (the notification center polls it — no new UI plumbing). Noise policy: WARN/FAIL always notify; a clean run notifies only on the daily post-close.

4. File placement map

4.1 New files (create)

Path	Size	Purpose
`api/intelligence/memory.ts`	~2.8 KB	Conversation memory: load/save/prune/clear per-user chat history.
`api/engine/learning.ts`	~10 KB	Learning loop: outcome recording, aggregation, mistake classification, divergence flags, lessons.
`api/systemcheck/checks.ts`	~17 KB	The 11 component probes, incl. the signal-stack self-test.
`api/systemcheck/service.ts`	~10 KB	Runner, AI diagnosis, admin notifications, ET scheduler.
`api/systemcheck-router.ts`	~0.7 KB	Owner-only tRPC routes: latest / history / runNow.
`src/pages/app/owner/SystemCheck.tsx`	~9 KB	Owner console page: live probes, AI diagnosis, history, scheduler status.

4.2 Edited files (apply diffs from appendices)

Path	Change
`api/intelligence/tools.ts`	Imports; LEARNING snapshot section; `getLearningReport` tool; history injection + persistence in `agentChat`; model default → `gpt-5.6`; addendum rule 7; backtest tool records outcomes. *(Appendix C)*
`api/intelligence-router.ts`	`history` + `clearHistory` routes. *(Appendix D)*
`api/engine-router.ts`	Backtest records outcomes; new `learning` query. *(Appendix E)*
`api/router.ts`	Register `systemCheck` router. *(Appendix I)*
`api/boot.ts`	Start the System Check scheduler in the production block. *(Appendix I)*
`db/schema.ts`	Three new tables appended. *(Appendix J)*
`api/lib/ensure-schema.ts`	Three CREATE TABLE statements + USER_ID_TABLES update. *(Appendix K)*
`src/App.tsx`	Route `/app/owner/system-check`. *(Appendix M)*
`src/pages/app/AppLayout.tsx`	Owner nav item. *(Appendix M)*

4.3 Dependency order

schema → ensure-schema → memory + learning (leaf modules) → tools.ts edits (imports both) → routers → systemcheck (imports engine + marketdata + queries) → router.ts → boot.ts → UI. The learning module imports the backtest *types only* — there is no circular dependency with the engine.

5. Integration steps

Follow in order. Each step names the appendix holding the code. The full updated source tree is also provided alongside this guide — the appendices exist so every change is reviewable line-by-line.

Step 1 — Database schema (Appendices J, K)

1. Append the three table definitions from Appendix J to `db/schema.ts` (after the existing tables).
2. Add the three `CREATE TABLE IF NOT EXISTS` statements from Appendix K to the `STATEMENTS` array in `api/lib/ensure-schema.ts`, and add the two user-scoped tables to `USER_ID_TABLES` (used by duplicate-user merges).
3. No manual migration needed — boot runs `ensureSchema()` idempotently. Expect the boot log to move from "14 tables" to "15 tables" (12 pre-existing + 3 new).

Step 2 — Conversation memory (Appendix A)

1. Create `api/intelligence/memory.ts` from Appendix A.
2. It imports `getDb` from `api/queries/connection` and `chatMessages` from `@db/schema` — both already exist in your tree.

Step 3 — Learning loop (Appendix B)

1. Create `api/engine/learning.ts` from Appendix B.
2. It imports backtest *types* from `api/engine/backtest.ts` (already in your tree from the engine add-on) — types only, no runtime cycle.

Step 4 — Intelligence wiring (Appendices C, D, E)

1. Apply the `tools.ts` edits from Appendix C — imports, LEARNING snapshot section, the `getLearningReport` tool definition and dispatch case, history injection + save in `agentChat`, the model default change, addendum rule 7, and outcome recording in the `runBacktest` tool case.
2. Apply the `intelligence-router.ts` edits from Appendix D (`history` + `clearHistory`).
3. Apply the `engine-router.ts` edits from Appendix E (record outcomes in `backtest`; add `learning` query). Note the backtest handler signature gains `ctx`.

Step 5 — System Check module (Appendices F, G, H)

1. Create the directory `api/systemcheck/` and add `checks.ts` (Appendix F) and `service.ts` (Appendix G).
2. Create `api/systemcheck-router.ts` (Appendix H). It uses `adminQuery` — owner-only by role.

Step 6 — Registration and boot (Appendix I)

1. Register the router in `api/router.ts`.

2. Add `ensureSystemCheckLoop()` to the production block of `api/boot.ts`, after governance init. It is a 60-second interval timer with `.unref()` — it cannot keep the process alive or block shutdown.

Step 7 — Owner console UI (Appendices L, M)

1. Create `src/pages/app/owner/SystemCheck.tsx` (Appendix L). It uses the shared owner components (`Badge`, `PageHeader`, `StatCard`, `TableShell`) already in your tree.
2. Add the route and the nav item (Appendix M).

Step 8 — Build, verify, deploy

1. `npx tsc --noEmit` — must be clean.
2. `npm run build` — vite client + esbuild server bundle.
3. Start with `NODE_ENV=production node dist/boot.js`. Watch the boot log: "schema ensured (15 tables)" then "Server running".
4. Smoke test: `GET /api/trpc/systemCheck.latest` unauthenticated → **401** (route exists, auth-gated). An unknown route → 404. Then sign in as the owner, open **Owner → System Check**, click **Run now**, and confirm the report renders and a notification appears in the bell.
5. Intelligence smoke test: send a chat, then a follow-up referencing it ("what did I just ask?") — memory working means the follow-up resolves correctly.

> **Deployment note:** the server only listens when `NODE_ENV=production`. Without it, `node dist/boot.js` exits silently — this has bitten us before. The scheduled checks (09:00 / 16:15 ET) only run in production mode.

6. Module behavior

6.1 Conversation memory

- **Storage:** `chat_messages` — userId, role (`user` | `assistant`), content (capped at 8,000 chars), createdAt. Only final user text and final assistant text are stored — tool-call scaffolding is deliberately excluded so future context stays clean.
- **Injection:** `agentChat` places history *after* the system prompt and *before* the new user message: `[system, …history, user]` . Last 20 messages (≈10 exchanges).
- **Retention:** 400 rows per user; pruned on write. Bounded cost, bounded table.
- **Failure posture:** every memory call is wrapped — a DB hiccup returns empty history or drops a save; the chat itself never fails because of memory.
- **User control:** `intelligence.history` (view, last 50) and `intelligence.clearHistory` (wipe — per-user only).
- **Authority:** memory is informational. It cannot satisfy a CONFIRM, cannot stage a ticket, cannot alter a clamp.

6.2 Learning loop (the “machine learning” component)

Honest framing for the team: this is not model training and does not reweight any neural network. It is a deterministic feedback layer — the mechanism that actually makes a rule-based trading system improve. It does three things:

- **Record** — `trade_outcomes` rows carry: symbol, strategyId, variantId, source (BACKTEST / SIMULATION / PAPER / LIVE), entry, exitPrice, qty, pnl, rMultiple, exitReason, and an auto-classified `lesson` . Recording happens automatically on every backtest (UI endpoint and AI tool), and is designed to accept paper/live outcomes as those paths close trades.
- **Diagnose** — `getLearningReport(userId)` returns: per-variant stats (trades, win rate, expectancy R, profit factor, total P&L, sorted by expectancy); **divergence flags** (live |expectancy − backtest expectancy| > 0.5R with ≥5 live and ≥10 backtest trades); **mistake patterns** (losers classified as stopped_out / stalled_no_followthrough / faded_into_close / other, with shares); the weakest variant (≥5 trades, negative expectancy); and distilled human-readable lessons with concrete suggestions (e.g. “consider raising min_signals_armed” when stalls dominate).
- **Feed back** — top-5 lessons are injected into every Intelligence snapshot under `LEARNING` , labeled as “hard-won facts about OUR system”. The AI is instructed (addendum rule 7) to apply them proactively and to call `getLearningReport` when asked what has been learned.

Failure posture: recording is fire-and-forget — a learning write failure returns `{recorded: 0}` and never fails a backtest. The report endpoint returns an empty-but-valid report when the table is empty or unreachable.

6.3 Model upgrade to GPT-5.6

- One-line default change in `tools.ts` : `process.env.OPENAI_MODEL ?? "gpt-5.6"` . The System Check diagnosis and LLM probe read the same env var.

- `gpt-5.6` aliases to the flagship Sol tier. Cost-aware tiers: `gpt-5.6-terra` (balanced) or `gpt-5.6-luna` (fast/cheap) can be set via `OPENAI_MODEL` without any code change.
- No other call-site changes: the chat-completions payload (tools, tool_choice, max_tokens) is unchanged.

6.4 System Check module

The 11 probes (each returns status + detail + remediation hint):

Probe	What it verifies	FAIL / WARN conditions
Environment & secrets	APP_ID, DATABASE_URL, GOVERNANCE_VAULT_KEY present; optional vars noted.	FAIL on missing required; WARN on missing OPENAI_API_KEY / IBKR_GATEWAY_URL.
Database	Live select on users + chat_messages + trade_outcomes.	FAIL on any error (points at ensureSchema logs).
Governance package	An ACTIVE signed package exists.	FAIL otherwise (engine would throw GOVERNANCE_UNAVAILABLE).
Market data feed	Gateway health, market open/closed, loop state, per-symbol errors, tracked count.	FAIL if gateway down with symbols tracked; WARN on symbol errors / empty watchlist. **Auto-remediates** a stopped loop during market hours.
Engine configs	Every builtin config loads through governance clamping.	FAIL on any load error.
Signal stack self-test	Trigger count ≥9; vwap_reclaim arms on a synthetic textbook reclaim; does NOT arm without a prior dip; sizePosition respects the $500 risk and 25% position ceilings.	FAIL on any assertion — "do not trust live proposals until fixed".
Autonomous monitors	Enumerates admin users' monitors and state breakdown.	WARN on KILLED monitors; OK when idle (valid state).
Ticket gate	Ticket store readable; state distribution; stale READY_FOR_CONFIRMATION past expiry.	WARN on stale tickets; FAIL on store error (critical path).
Intelligence (LLM)	Live 8-token ping to the configured model with latency.	WARN without API key; FAIL on HTTP/network error.
Memory	chat_messages readable.	FAIL on error.
Learning loop	trade_outcomes readable and non-empty.	WARN when empty (nothing to learn from yet); FAIL on error.

AI diagnosis: on any WARN/FAIL — and on every post-close run — the issues go to GPT-5.6 with a reliability-engineer system prompt that includes the architecture summary and a hard rule: *never suggest loosening risk limits, bypassing governance, or auto-executing without confirmation*. Output sections: BROKEN (what/cause/exact fix), PREVENT (systemic fix per failure class), GROWTH (2–3 weekly improvements). Diagnosis failure (no key, model error) degrades to the deterministic remediation hints — never blocks the report.

Noise policy: WARN/FAIL always notify all admin users (CRITICAL/HIGH priority, via the alerts table the notification center already polls); a clean run notifies only on the daily post-close (LOW “all clear”); pre-open and manual clean runs stay silent.

Scheduler: 60s tick, ET time via `Intl.DateTimeFormat` (America/New_York), Mon–Fri only; windows 09:00–09:25 and 16:15–16:40 with per-day latches; `.unref()`ed timer. Concurrency-guarded (a manual run during a scheduled run returns the latest report rather than double-running).

7. Operations — environment, schedule, failure modes

7.1 Environment variables

Variable	Required	Effect in this version
`OPENAI_API_KEY`	Optional	Enables Intelligence reasoning loop AND System Check AI diagnosis. Without it: chat falls back to canned answers; System Check still runs all probes and reports with deterministic remediation hints (WARN on the LLM probe).
`OPENAI_MODEL`	Optional	Overrides the `gpt-5.6` default everywhere (chat, diagnosis, LLM probe). Tiers: `gpt-5.6` (Sol, flagship), `gpt-5.6-terra` (balanced), `gpt-5.6-luna` (fast/cheap).
`GOVERNANCE_VAULT_KEY`	**Required**	Missing ⇒ governance probe FAILS and engine config loading throws by design.
`DATABASE_URL`	**Required**	Missing ⇒ database probe FAILS; memory/learning degrade silently in chat but reports will show it.
`IBKR_GATEWAY_URL`	Optional	Defaults to localhost gateway; WARN from the feed probe in deployments without a gateway.
`NODE_ENV`	**Required**	Must be `production` — otherwise the server never listens and the scheduler never starts.

7.2 Schedule (all times America/New_York, Mon–Fri)

Run	Window	Behavior
Pre-open	09:00–09:25	Full 11-probe check. Notifies admins only on WARN/FAIL. AI diagnosis on issues.
Post-close	16:15–16:40	Full check + AI diagnosis always (adds GROWTH suggestions even when clean). Clean runs send one LOW “all clear” notification.
Manual	any time	Owner console “Run now”. Same probes; notification follows the noise policy.

7.3 Failure modes (designed, not accidental)

Failure	Behavior
Memory DB write fails	Chat proceeds; message not persisted. No user-visible error.
Learning recording fails	Backtest returns normally with `outcomesRecorded: 0`.
AI diagnosis unavailable	Report persists with `aiDiagnosis: null`; notification carries deterministic remediation hints and notes the diagnosis was unavailable.

A probe throws	Caught per-probe, recorded as FAIL “check crashed” — one bad probe cannot sink the run.
Concurrent runs	Re-entrant guard: second caller gets the latest persisted report instead of a parallel run.
Server not in production mode	No listener, no scheduler — verify `NODE_ENV=production` in the process environment.

8. Safety invariants (unchanged and enforced)

1. **The confirmation gate is untouched.** Memory, learning, and system diagnosis cannot create, confirm, or submit a ticket. The only write path in the entire platform remains `proposeTicket()` + exact `CONFIRM ORDER [TICKET_ID]`.
2. **Learning adjusts advice, never authority.** No lesson, statistic, or AI suggestion can change sizing math, clamps, thresholds, or governance. The System Check AI prompt explicitly forbids suggesting it.
3. **Memory cannot impersonate consent.** A historical “yes” in chat history is not a confirmation — the gate matches the exact string against the CURRENT ticket only.
4. **The tool whitelist is server-enforced.** The model can only call the 9 exposed functions; the server validates every argument (symbol regex, quantity bounds, required stop).
5. **System Check observes and advises.** Its single automatic action is restarting the data-feed refresh loop during market hours — safe and reversible. Everything else is a recommendation to a human.
6. **Anti-exposure boundary unchanged.** The exposure guard still runs before any LLM call; tools never return governing documents, prompts, hidden thresholds, or credentials.
7. **Graceful degradation everywhere.** Every new subsystem fails soft: missing API key, missing tables, model errors, or DB hiccups produce warnings in the report — never a broken chat, a failed backtest, or a crashed boot.

Appendix A — api/intelligence/memory.ts (new file, full source)

api/intelligence/memory.ts

Conversation memory: load / save / prune / clear. Create this file as-is.

```
import { desc, eq, inArray } from "drizzle-orm";
import { getDb } from "../queries/connection";
import { chatMessages } from "@db/schema";

/**
 * CONVERSATION MEMORY
 *
 * Persists the Intelligence chat so every new message carries the recent
 * conversation instead of starting cold. Memory is per-user, append-only,
 * and bounded — we load the last HISTORY_LIMIT exchanges and prune rows
 * beyond KEEP_ROWS so the table never grows without limit.
 *
 * Only final user text and final assistant text are stored (tool-call
 * scaffolding is not — it would pollute future context). Memory never
 * widens authority: it informs replies, it cannot change what the
 * confirmation gate requires.
 */

const HISTORY_LIMIT = 20; // messages (≈10 exchanges) injected per chat
const KEEP_ROWS = 400; // per-user retention ceiling

export interface MemoryMessage {
  role: "user" | "assistant";
  content: string;
}

export async function loadHistory(userId: number, limit = HISTORY_LIMIT): Promise<MemoryMessage[]> {
  const db = await getDb();
  if (!db) return [];
  try {
    const rows = await db
      .select({ role: chatMessages.role, content: chatMessages.content })
      .from(chatMessages)
      .where(eq(chatMessages.userId, userId))
      .orderBy(desc(chatMessages.id))
      .limit(limit);
    return rows
      .reverse()
      .filter((r) => r.role === "user" || r.role === "assistant")
      .map((r) => ({ role: r.role as "user" | "assistant", content: r.content }));
  } catch {
    return []; // memory is an enhancement — never block a chat on it
  }
}

export async function saveMessage(userId: number, role: "user" | "assistant", content: string):
Promise<void> {
  const db = await getDb();
  if (!db) return;
  const trimmed = content.trim();
  if (!trimmed) return;
  try {
    await db.insert(chatMessages).values({ userId, role, content: trimmed.slice(0, 8000) });
    await prune(userId);
  } catch {
    /* swallow — a memory write failure must not break the chat */
  }
```

```
}

async function prune(userId: number): Promise<void> {
  const db = await getDb();
  if (!db) return;
  const rows = await db
    .select({ id: chatMessages.id })
    .from(chatMessages)
    .where(eq(chatMessages.userId, userId))
    .orderBy(desc(chatMessages.id))
    .limit(1000);
  if (rows.length <= KEEP_ROWS) return;
  const cutoff = rows[KEEP_ROWS - 1].id;
  const ids = rows.filter((r) => r.id < cutoff).map((r) => Number(r.id));
  if (ids.length === 0) return;
  await db.delete(chatMessages).where(inArray(chatMessages.id, ids));
}

export async function clearHistory(userId: number): Promise<{ cleared: boolean }> {
  const db = await getDb();
  if (!db) return { cleared: false };
  await db.delete(chatMessages).where(eq(chatMessages.userId, userId));
  return { cleared: true };
}
```

Appendix B — api/engine/learning.ts (new file, full source)

api/engine/learning.ts

Learning loop: record, diagnose, feed back. Create this file as-is.

```
import { desc, eq } from "drizzle-orm";
import { getDb } from "../queries/connection";
import { tradeOutcomes, type TradeOutcome } from "@db/schema";
import type { BacktestReport, BacktestTrade } from "./backtest";

/**
 * LEARNING LOOP — "learn from mistakes," implemented honestly.
 *
 * This is NOT neural-network training and it does not reweight any model.
 * It is a deterministic feedback layer that does three things:
 *
 *   1. RECORD — every completed trade (backtest today; paper/live as those
 *      paths produce closed outcomes) is logged with its full decision
 *      context: strategy, variant, entry/exit, R-multiple, exit reason.
 *   2. DIAGNOSE — outcomes are aggregated into per-variant performance and
 *      classified mistakes (stop-outs, stalls, flatten losses), plus a
 *      divergence flag when live results drift from backtest expectations
 *      — the exact "if live results diverge from backtest, flag it" rule
 *      from the engine build spec.
 *   3. FEED BACK — the distilled lessons are injected into the Intelligence
 *      snapshot (so the AI advises from OUR outcomes, not generic theory)
 *      and exposed to the simulator so variant ranking can see live stats.
 *
 * What this deliberately does NOT do: change sizing, loosen clamps, or
 * alter governance. Learning adjusts RANKING and ADVICE. Authority stays
 * exactly where the constitution put it.
 */

/* ---------- 1. RECORD ---------- */

function classifyLesson(t: BacktestTrade): string {
  const reason = t.exitReason.toLowerCase();
  if (t.r > 0) {
    if (reason.includes("target")) return "plan worked: target ladder paid";
    if (reason.includes("flatten")) return "held to close: modest gain, no target hit";
    return "winner: " + t.exitReason;
  }
  if (reason.includes("stop")) return "mistake candidate: stopped out — entry quality or stop
placement";
  if (reason.includes("stall") || reason.includes("timer")) return "mistake candidate: stalled —
trigger fired but no follow-through";
  if (reason.includes("flatten")) return "mistake candidate: faded into the close — late entry or
weak trend";
  if (reason.includes("chase")) return "avoided: chase-cancel invalidated entry";
  return "loser: " + t.exitReason;
}

export async function recordBacktestOutcomes(
  userId: number,
  symbol: string,
  strategyId: string,
  report: BacktestReport
): Promise<{ recorded: number }> {
  const db = await getDb();
  if (!db || report.trades.length === 0) return { recorded: 0 };
  const rows = report.trades.map((t) => ({
```

```
symbol,
    strategyId,
    variantId: t.variantId,
    source: "BACKTEST" as const,
    entry: String(t.entry),
    exitPrice: String(t.avgExit),
    qty: t.qty,
    pnl: String(t.pnl.toFixed(2)),
    rMultiple: String(t.r.toFixed(3)),
    exitReason: t.exitReason,
    lesson: classifyLesson(t),
  }));
  try {
    await db.insert(tradeOutcomes).values(rows);
    return { recorded: rows.length };
  } catch {
    return { recorded: 0 }; // learning is additive — never fail a backtest over it
  }
}

/* ---------- 2. DIAGNOSE ---------- */

export interface VariantStats {
  strategyId: string;
  variantId: string | null;
  source: string;
  trades: number;
  wins: number;
  winRate: number | null;
  expectancyR: number | null;
  profitFactor: number | null;
  totalPnl: number;
}

export interface DivergenceFlag {
  strategyId: string;
  variantId: string | null;
  backtestExpectancyR: number;
  liveExpectancyR: number;
  liveTrades: number;
  message: string;
}

export interface LearningReport {
  totalOutcomes: number;
  byVariant: VariantStats[];
  divergences: DivergenceFlag[];
  mistakePatterns: { category: string; count: number; shareOfLosses: number }[];
  worstVariant: { strategyId: string; variantId: string | null; expectancyR: number; trades: number }
| null;
  lessons: string[];
}

function aggregate(rows: TradeOutcome[]): VariantStats[] {
  const groups = new Map<string, TradeOutcome[]>();
  for (const r of rows) {
    const key = `${r.strategyId}|${r.variantId ?? "-"}|${r.source}`;
    const arr = groups.get(key) ?? [];
    arr.push(r);
    groups.set(key, arr);
  }
  const stats: VariantStats[] = [];
  for (const [key, arr] of groups) {
    const [strategyId, variantId, source] = key.split("|");
    const rs = arr.map((r) => Number(r.rMultiple ?? 0));
```

```
const wins = arr.filter((r) => Number(r.pnl ?? 0) > 0);
    const grossWin = wins.reduce((a, r) => a + Number(r.pnl ?? 0), 0);
    const grossLoss = Math.abs(arr.filter((r) => Number(r.pnl ?? 0) <= 0).reduce((a, r) => a +
Number(r.pnl ?? 0), 0));
    stats.push({
      strategyId,
      variantId: variantId === "-" ? null : variantId,
      source,
      trades: arr.length,
      wins: wins.length,
      winRate: arr.length > 0 ? +((wins.length / arr.length) * 100).toFixed(1) : null,
      expectancyR: arr.length > 0 ? +(rs.reduce((a, b) => a + b, 0) / arr.length).toFixed(3) : null,
      profitFactor: grossLoss > 0 ? +(grossWin / grossLoss).toFixed(2) : grossWin > 0 ? Infinity :
null,
      totalPnl: +pnls.reduce((a, b) => a + b, 0).toFixed(2),
    });
  }
  return stats.sort((a, b) => (b.expectancyR ?? -99) - (a.expectancyR ?? -99));
}

function mistakeCategory(outcome: TradeOutcome): string | null {
  if (Number(outcome.pnl ?? 0) > 0) return null;
  const reason = (outcome.exitReason ?? "").toLowerCase();
  if (reason.includes("stop")) return "stopped_out";
  if (reason.includes("stall") || reason.includes("timer")) return "stalled_no_followthrough";
  if (reason.includes("flatten")) return "faded_into_close";
  return "other_loss";
}

export async function getLearningReport(userId: number): Promise<LearningReport> {
  const empty: LearningReport = {
    totalOutcomes: 0, byVariant: [], divergences: [], mistakePatterns: [], worstVariant: null,
    lessons: ["No outcomes logged yet — run a backtest to start the learning loop."],
  };
  const db = await getDb();
  if (!db) return empty;
  let rows: TradeOutcome[];
  try {
    rows = await db
      .select()
      .from(tradeOutcomes)
      .where(eq(tradeOutcomes.userId, userId))
      .orderBy(desc(tradeOutcomes.id))
      .limit(5000);
  } catch {
    return empty;
  }
  if (rows.length === 0) return empty;

  const byVariant = aggregate(rows);

  // Divergence: live (PAPER/LIVE) expectancy vs backtest expectancy, same strategy+variant
  const divergences: DivergenceFlag[] = [];
  for (const live of byVariant.filter((s) => (s.source === "PAPER" || s.source === "LIVE") &&
s.trades >= 5)) {
    const bt = byVariant.find(
      (s) => s.source === "BACKTEST" && s.strategyId === live.strategyId && s.variantId ===
live.variantId && s.trades >= 10
    );
    if (!bt || bt.expectancyR == null || live.expectancyR == null) continue;
    if (Math.abs(live.expectancyR - bt.expectancyR) > 0.5) {
      divergences.push({
        strategyId: live.strategyId,
        variantId: live.variantId,
        backtestExpectancyR: bt.expectancyR,
```

```
liveTrades: live.trades,
        message:
          `DIVERGENCE: ${live.strategyId}/${live.variantId ?? "?"} backtest expectancy
${bt.expectancyR}R ` +
          `but live is ${live.expectancyR}R over ${live.trades} trades — live results diverge from
backtest. Investigate before trusting this variant.`,
      });
    }
  }

  // Mistake patterns across all losers
  const losers = rows.filter((r) => Number(r.pnl ?? 0) <= 0);
  const catCounts = new Map<string, number>();
  for (const r of losers) {
    const c = mistakeCategory(r);
    if (c) catCounts.set(c, (catCounts.get(c) ?? 0) + 1);
  }
  const mistakePatterns = [...catCounts.entries()]
    .map(([category, count]) => ({ category, count, shareOfLosses: losers.length > 0 ? +((count /
losers.length) * 100).toFixed(1) : 0 }))
    .sort((a, b) => b.count - a.count);

  // Worst variant with a meaningful sample
  const worstVariant =
    byVariant
      .filter((s) => s.trades >= 5 && s.expectancyR != null && s.expectancyR < 0)
      .sort((a, b) => (a.expectancyR ?? 0) - (b.expectancyR ?? 0))[0] ?? null;

  // Distilled lessons (human- and LLM-readable)
  const lessons: string[] = [];
  for (const d of divergences) lessons.push(d.message);
  if (mistakePatterns[0] && losers.length >= 5) {
    lessons.push(
      `Most common mistake: ${mistakePatterns[0].category} — ${mistakePatterns[0].count} of
${losers.length} losing trades (${mistakePatterns[0].shareOfLosses}%). ` +
        (mistakePatterns[0].category === "stopped_out"
          ? "Consider whether entries are chasing or stops are too tight for current ATR."
          : mistakePatterns[0].category === "stalled_no_followthrough"
            ? "Triggers fire but moves fail — consider raising min_signals_armed or the volume
threshold."
            : "Entries may be too late in the session — check the no-entries-after cutoff.")
    );
  }
  if (worstVariant) {
    lessons.push(
      `Weakest variant: ${worstVariant.strategyId}/${worstVariant.variantId ?? "?"} — expectancy
${worstVariant.expectancyR}R over ${worstVariant.trades} trades. Deprioritize it in the ladder.`
    );
  }
  const best = byVariant.find((s) => s.trades >= 5 && (s.expectancyR ?? -99) > 0);
  if (best) {
    lessons.push(`Best performer: ${best.strategyId}/${best.variantId ?? "?"} (${best.source}) —
expectancy ${best.expectancyR}R, win rate ${best.winRate}%.`);
  }
  if (lessons.length === 0) lessons.push(`${rows.length} outcomes logged; no strong patterns yet —
keep collecting.`);

  return { totalOutcomes: rows.length, byVariant: byVariant.slice(0, 30), divergences,
mistakePatterns, worstVariant, lessons };
}

/* ---------- 3. FEED BACK (into the Intelligence snapshot) ---------- */

export async function learningSummaryLines(userId: number): Promise<string[]> {
  const report = await getLearningReport(userId).catch(() => null);
```

```
return report.lessons.slice(0, 5);
}
```

Appendix C — api/intelligence/tools.ts (edits to the existing file)

Seven edits. Apply them in order; each snippet shows enough context to locate the insertion point.

C1 — imports (top of file)

Add the last two import lines.

```
import { scanFeeds } from "../engine/scanner";
import { SWARM_MASTER_PROMPT } from "../prompts/swarm-master";
import { loadHistory, saveMessage } from "./memory";
import { getLearningReport, learningSummaryLines, recordBacktestOutcomes } from "../engine/learning";
```

C2 — buildSnapshot(): LEARNING section

Replace the final two lines of buildSnapshot (the STRATEGY CONFIGS push and return) with this block.

```
  sections.push(`STRATEGY CONFIGS AVAILABLE: ${BUILTIN_CONFIGS.map((c) => `${c.strategy_id}
("${c.label}")`).join(", ")}`);

  try {
    const lessons = await learningSummaryLines(userId);
    if (lessons.length > 0) {
      sections.push(
        "LEARNING (distilled from our logged trade outcomes — treat these as hard-won facts about OUR
system, not generic theory):\n" +
          lessons.map((l) => `- ${l}`).join("\n")
      );
    }
  } catch {
    /* learning section optional */
  }

  return sections.join("\n");
```

C3 — TOOL_DEFS: add getLearningReport

Insert the second entry immediately after the getGovernanceStatus definition.

```
  { type: "function", function: { name: "getGovernanceStatus", description: "Get the active signed
governance package version and integrity status.", parameters: { type: "object", properties: {} } }
},
  { type: "function", function: { name: "getLearningReport", description: "Get the learning-loop
report: per-variant performance from logged outcomes, classified mistake patterns (stop-outs, stalls,
fades), divergence flags where live results drift from backtest, and distilled lessons. Use when
asked what the system has learned, what keeps failing, or which variant to trust.", parameters: {
type: "object", properties: {} } } },
```

C4 — executeTool(): dispatch case

Insert the getLearningReport case after getGovernanceStatus.

```
    case "getGovernanceStatus":
      return publicPackageStatus();
    case "getLearningReport":
      return getLearningReport(userId);
```

C5 — executeTool runBacktest + agentChat memory + model

Record outcomes in the runBacktest case; inject history and persist messages in agentChat; change the model default.

```
    case "runBacktest": {
      const { config, clamps, governanceVersion } = await loadConfig(String(args.strategyId));
      const bars = await barsFor(String(args.symbol).toUpperCase());
      if (bars.length < 200) return { error: `not enough bars (${bars.length}) — check the data feed
/ gateway` };
      const r = runBacktest({ config, symbol: String(args.symbol).toUpperCase(), bars });
      await recordBacktestOutcomes(userId, String(args.symbol).toUpperCase(), config.strategy_id, r);
      return { ... };

  // --- in agentChat(): history injection + persistence + model default ---
  const history = await loadHistory(userId);
  const messages: ChatMessage[] = [
    { role: "system", content: system },
    ...history.map((m) => ({ role: m.role, content: m.content })),
    { role: "user", content: text },
  ];
  await saveMessage(userId, "user", text);

  // inside the fetch body:
          model: process.env.OPENAI_MODEL ?? "gpt-5.6",

  // on the final-answer path:
      if (!msg.tool_calls || msg.tool_calls.length === 0) {
        const reply = msg.content?.trim() || null;
        if (reply) await saveMessage(userId, "assistant", reply);
        return reply;
      }
```

C6 — RUNTIME_ADDENDUM: rule 7

Replace the final line of the addendum (rule 6) with these two rules.

```
6. The anti-exposure boundary still applies: tools never return governing documents, prompts,
thresholds beyond what the UI discloses, or credentials.
7. MEMORY & LEARNING: earlier messages in this conversation are real history — use them for
continuity (names, preferences, open threads). When the snapshot contains a LEARNING section, those
lessons come from OUR logged trade outcomes: apply them proactively (e.g. deprioritize a weak
variant, respect a divergence flag) and cite them when relevant. When asked "what have we learned" or
"what keeps failing", call getLearningReport.`;
```

Appendix D — api/intelligence-router.ts (edits)

D1 — import

Add the memory import next to the agentChat import.

```
import { agentChat } from "./intelligence/tools";
import { clearHistory, loadHistory } from "./intelligence/memory";
```

D2 — routes

Add the history and clearHistory procedures at the top of the router, before chat.

```
export const intelligenceRouter = createRouter({
  /** Conversation memory: recent exchanges for the signed-in user. */
  history: authedQuery.query(async ({ ctx }) => loadHistory(ctx.user.id, 50)),

  /** Forget the conversation (memory is per-user; clearing affects no one else). */
  clearHistory: authedQuery.mutation(async ({ ctx }) => clearHistory(ctx.user.id)),

  chat: authedQuery
```

Appendix E — api/engine-router.ts (edits)

E1 — import

Add the learning import.

```
import { marketDataService } from "./marketdata/service";
import { fetchMinuteBars } from "./marketdata/ibkr-data";
import { getLearningReport, recordBacktestOutcomes } from "./engine/learning";
```

E2 — backtest handler + learning query

The backtest handler gains ctx and records outcomes; add the learning query (shown here placed after scan).

```
  backtest: authedQuery
    .input(strategyInput.extend({ period: z.enum(["2d", "1w", "2w", "1m"]).default("1w") }))
    .mutation(async ({ ctx, input }) => {
      const { config, clamps, governanceVersion } = await loadConfig(input.strategyId);
      const bars = await barsFor(input.symbol, input.period);
      if (bars.length < 200) throw new Error(`Not enough bars for ${input.symbol.toUpperCase()}
(${bars.length}) — check the data feed`);
      const report = runBacktest({ config, symbol: input.symbol.toUpperCase(), bars });
      // Learning loop: every backtest's outcomes feed the mistake/lesson layer.
      const { recorded } = await recordBacktestOutcomes(ctx.user.id, input.symbol.toUpperCase(),
config.strategy_id, report);
      return { ...report, events: report.events.slice(-30), governanceVersion, clamps,
outcomesRecorded: recorded };
    }),

  /** Learning loop report: per-variant stats, mistake patterns, divergences, lessons. */
  learning: authedQuery.query(async ({ ctx }) => getLearningReport(ctx.user.id)),
```

Appendix F — api/systemcheck/checks.ts (new file, full source)

api/systemcheck/checks.ts

The 11 component probes. Create directory api/systemcheck/ first, then this file.

```
import { desc, eq } from "drizzle-orm";
import { getDb } from "../queries/connection";
import { users, orderTickets, chatMessages, tradeOutcomes } from "@db/schema";
import { marketDataService } from "../marketdata/service";
import { publicPackageStatus } from "../governance/runtime";
import { BUILTIN_CONFIGS, loadConfig } from "../engine/config";
import { TRIGGERS, runTrigger } from "../engine/triggers";
import { sizePosition } from "../engine/risk";
import { listMonitors } from "../engine/monitor";
import type { Bar } from "../marketdata/indicators";

/**
 * SYSTEM CHECK — component health probes.
 *
 * Each check is a real, executable probe — not a ping that always passes.
 * The signal check, for example, feeds synthetic bars through the actual
 * trigger functions and asserts the expected output: it proves the signal
 * math FIRES, not merely that the file imports.
 *
 * Every check returns a remediation hint so failures arrive with a fix,
 * and the AI diagnosis layer (service.ts) turns those into full
 * what-broken / why / how-to-fix / how-to-improve guidance.
 */

export interface CheckResult {
  id: string;
  name: string;
  status: "OK" | "WARN" | "FAIL";
  detail: string;
  remediation?: string;
}

type CheckFn = () => Promise<CheckResult>;

function ok(id: string, name: string, detail: string): CheckResult {
  return { id, name, status: "OK", detail };
}
function warn(id: string, name: string, detail: string, remediation: string): CheckResult {
  return { id, name, status: "WARN", detail, remediation };
}
function fail(id: string, name: string, detail: string, remediation: string): CheckResult {
  return { id, name, status: "FAIL", detail, remediation };
}

/* ---------- 1. environment ---------- */

const checkEnvironment: CheckFn = async () => {
  const id = "environment";
  const name = "Environment & secrets";
  const missing: string[] = [];
  if (!process.env.APP_ID) missing.push("APP_ID");
  if (!process.env.DATABASE_URL) missing.push("DATABASE_URL");
  if (!process.env.GOVERNANCE_VAULT_KEY) missing.push("GOVERNANCE_VAULT_KEY");
  if (missing.length > 0) {
    return fail(id, name, `Missing required env vars: ${missing.join(", ")} — governance and/or
database will refuse to operate`,
```

```
}
  const warnings: string[] = [];
  if (!process.env.OPENAI_API_KEY) warnings.push("OPENAI_API_KEY not set — Intelligence runs in
fallback mode");
  if (!process.env.IBKR_GATEWAY_URL) warnings.push("IBKR_GATEWAY_URL not set — data feed targets
localhost default");
  if (warnings.length > 0) {
    return warn(id, name, warnings.join(" · "),
      "Set the listed variables in deployment. OPENAI_API_KEY restores full Intelligence;
IBKR_GATEWAY_URL points the feed at the real Client Portal gateway.");
  }
  return ok(id, name, "All required and optional env vars present");
};

/* ---------- 2. database ---------- */

const checkDatabase: CheckFn = async () => {
  const id = "database";
  const name = "Database";
  try {
    const db = getDb();
    await db.select({ id: users.id }).from(users).limit(1);
    // verify the newer tables this module depends on
    await db.select({ id: chatMessages.id }).from(chatMessages).limit(1);
    await db.select({ id: tradeOutcomes.id }).from(tradeOutcomes).limit(1);
    return ok(id, name, "Connection healthy; core + memory + learning tables present");
  } catch (e) {
    return fail(id, name, `Database error: ${(e as Error).message}`,
      "Check DATABASE_URL and that the MySQL/TiDB instance is reachable from this deployment. Boot
runs ensureSchema() — a missing table here means boot schema creation failed; check server logs.");
  }
};

/* ---------- 3. governance ---------- */

const checkGovernance: CheckFn = async () => {
  const id = "governance";
  const name = "Governance package";
  try {
    const pkgs = await publicPackageStatus();
    const active = pkgs.find((p) => p.status === "ACTIVE");
    if (!active) {
      return fail(id, name, "NO ACTIVE governance package — engine config loading will throw
GOVERNANCE_UNAVAILABLE",
        "Boot should stage a genesis package via ensureGovernanceReady(). Check GOVERNANCE_VAULT_KEY
is set and inspect governance router status; activate a signed package manually if needed.");
    }
    return ok(id, name, `Package ${active.version} ACTIVE (hash ${active.hashShort})`);
  } catch (e) {
    return fail(id, name, `Governance status error: ${(e as Error).message}`,
      "Check the governance runtime and vault key; the engine cannot run without an active signed
package.");
  }
};

/* ---------- 4. market data feed (with safe auto-remediation) ---------- */

const checkMarketData: CheckFn = async () => {
  const id = "market_data";
  const name = "Market data feed";
  try {
    const feed = await marketDataService.status();
    const problems: string[] = [];
    if (!feed.gatewayOk) problems.push(`gateway DOWN (${feed.gatewayDetail})`);
    if (feed.marketOpen && !feed.loopRunning && feed.symbols.length > 0) {
```

```
marketDataService.ensureLoop();
      problems.push("refresh loop was down during market hours — AUTO-REMEDIATED (restarted)");
    }
    const errored = feed.symbols.filter((s) => s.error);
    if (errored.length > 0) problems.push(`${errored.length} tracked symbol(s) erroring:
${errored.map((s) => s.symbol).join(", ")}`);
    if (feed.symbols.length === 0) problems.push("no symbols tracked — scanner and monitors have
nothing to evaluate");
    if (problems.length > 0) {
      const isFail = !feed.gatewayOk && feed.symbols.length > 0;
      const detail = problems.join(" · ");
      const status = isFail ? "FAIL" : "WARN";
      return (status === "FAIL" ? fail : warn)(id, name, detail,
        "Verify the IBKR Client Portal gateway is running and authenticated (GET
/iserver/auth/status). If the gateway is intentional (off-hours / dev), this is expected. Track
symbols via the Data Feed panel.");
    }
    return ok(id, name, `gateway OK · market ${feed.marketOpen ? "OPEN" : "CLOSED"} ·
${feed.symbols.length} symbol(s) tracked · loop ${feed.loopRunning ? "running" : "standby (off-
hours)"}`);
  } catch (e) {
    return fail(id, name, `Feed status error: ${(e as Error).message}`,
      "Inspect the marketdata service and gateway connectivity.");
  }
};

/* ---------- 5. engine configs (governance-clamped) ---------- */

const checkEngineConfigs: CheckFn = async () => {
  const id = "engine_configs";
  const name = "Engine configs & governance clamps";
  try {
    const loaded: string[] = [];
    for (const c of BUILTIN_CONFIGS) {
      const { config, clamps, governanceVersion } = await loadConfig(c.strategy_id);
      loaded.push(`${config.strategy_id} v${config.version} (${clamps.length} clamp(s), gov
${governanceVersion})`);
    }
    return ok(id, name, `${BUILTIN_CONFIGS.length} configs load under governance: ${loaded.join(" ·
")}`);
  } catch (e) {
    return fail(id, name, `Config load failed: ${(e as Error).message}`,
      "If this is GOVERNANCE_UNAVAILABLE, activate a governance package first. Otherwise inspect
engine/config.ts clamping against the active package ceiling.");
  }
};

/* ---------- 6. signals self-test (proves the math fires) ---------- */

function syntheticBar(t: number, c: number, v = 100): Bar {
  return { t, o: c, h: c + 0.1, l: c - 0.1, c, v };
}

const checkSignals: CheckFn = async () => {
  const id = "signals";
  const name = "Signal stack self-test";
  const problems: string[] = [];
  try {
    const triggerCount = Object.keys(TRIGGERS).length;
    if (triggerCount < 9) problems.push(`trigger registry has ${triggerCount} entries, expected ≥
9`);

    // Positive test: dip below VWAP then reclaim and hold → MUST arm
    const t0 = Date.UTC(2026, 0, 5, 14, 30); // fixed synthetic session
    const session: Bar[] = [
```

```
syntheticBar(t0 + 1 * 60000, 100),
      syntheticBar(t0 + 2 * 60000, 100),
      syntheticBar(t0 + 3 * 60000, 99), // below vwap
      syntheticBar(t0 + 4 * 60000, 102),
      syntheticBar(t0 + 5 * 60000, 102),
      syntheticBar(t0 + 6 * 60000, 102),
      syntheticBar(t0 + 7 * 60000, 102),
    ];
    const pos = runTrigger("vwap_reclaim", { session, priorDay: { high: 101, low: 98, close: 99.5 }
}, { hold_minutes: 3 });
    if (!pos.armed) problems.push(`vwap_reclaim FAILED to arm on a textbook reclaim pattern
(evidence: ${JSON.stringify(pos.evidence)})`);

    // Negative test: never below VWAP → MUST NOT arm (guards against always-on regression)
    const flat = Array.from({ length: 8 }, (_, i) => syntheticBar(t0 + i * 60000, 100 + i * 0.01));
    const neg = runTrigger("vwap_reclaim", { session: flat, priorDay: { high: 101, low: 98, close:
99.5 } }, { hold_minutes: 3 });
    if (neg.armed) problems.push("vwap_reclaim armed WITHOUT a prior dip — trigger logic regressed");

    // Risk sizing sanity: 0.5% of 100k = $500 risk, $1/share → 500 qty, capped by 25% position cap
    const sizing = sizePosition({ accountEquity: 100000, entry: 100, stop: 99, maxRiskPct: 0.5,
maxPositionPct: 25 });
    if (!(sizing.qty > 0)) problems.push("sizePosition returned zero qty on a valid setup");
    if (sizing.dollarRisk > 500.01) problems.push(`sizePosition risk $${sizing.dollarRisk} exceeds
the $500 ceiling — risk math regressed`);
    if (sizing.positionValue > 25000.01) problems.push(`sizePosition value $${sizing.positionValue}
exceeds the 25% position cap`);

    if (problems.length > 0) {
      return fail(id, name, problems.join(" · "),
        "Signal or risk math regressed — do NOT trust live proposals until fixed. Run engine.backtest
on a tracked symbol to reproduce, then inspect engine/triggers.ts and engine/risk.ts.");
    }
    return ok(id, name, `${triggerCount} triggers registered · reclaim self-test armed correctly ·
negative test held · risk sizing within ceilings (qty ${sizing.qty}, risk
$${sizing.dollarRisk.toFixed(0)})`);
  } catch (e) {
    return fail(id, name, `Signal self-test threw: ${(e as Error).message}`,
      "A trigger or risk function crashed on synthetic input — inspect engine/triggers.ts and
engine/risk.ts before trusting any proposal.");
  }
};

/* ---------- 7. autonomous monitors ---------- */

const checkAutonomous: CheckFn = async () => {
  const id = "autonomous";
  const name = "Autonomous monitors";
  try {
    const db = getDb();
    const admins = await db.select({ id: users.id }).from(users).where(eq(users.role, "admin"));
    let total = 0;
    const states = new Map<string, number>();
    for (const a of admins) {
      for (const m of listMonitors(a.id)) {
        total += 1;
        states.set(m.state, (states.get(m.state) ?? 0) + 1);
      }
    }
    if (total === 0) return ok(id, name, "No active monitors (idle is a valid state — monitors start
from the Engine panel or scanner)");
    const breakdown = [...states.entries()].map(([s, n]) => `${s}:${n}`).join(" · ");
    const killed = states.get("KILLED") ?? 0;
    if (killed > 0) {
      return warn(id, name, `${total} monitor(s): ${breakdown} — ${killed} KILLED monitor(s)
```

```
"Inspect killed monitors' last events (engine.monitors) — a kill means the state machine invalidated
the setup, which is working as designed, but repeated kills on the same symbol suggest a config/feed
issue.");
    }
    return ok(id, name, `${total} monitor(s) active: ${breakdown}`);
  } catch (e) {
    return fail(id, name, `Monitor check error: ${(e as Error).message}`,
      "Inspect engine/monitor.ts — monitors failing to enumerate means the autonomous path is
unhealthy.");
  }
};

/* ---------- 8. ticket gate ---------- */

const checkTicketGate: CheckFn = async () => {
  const id = "ticket_gate";
  const name = "Order ticket gate";
  try {
    const db = getDb();
    const recent = await db.select().from(orderTickets).orderBy(desc(orderTickets.id)).limit(50);
    const now = Date.now();
    const staleAwaiting = recent.filter(
      (t) => t.state === "READY_FOR_CONFIRMATION" && t.expiresAt && new Date(t.expiresAt).getTime() <
now - 10 * 60 * 1000
    );
    if (staleAwaiting.length > 0) {
      return warn(id, name, `${staleAwaiting.length} ticket(s) still READY_FOR_CONFIRMATION past
expiry — expiry sweeper may not be running`,
        "Expired tickets must never route. Verify the ticket expiry/sweep path in queries/tickets.ts;
stale rows should transition to EXPIRED.");
    }
    const byState = new Map<string, number>();
    for (const t of recent) byState.set(t.state, (byState.get(t.state) ?? 0) + 1);
    return ok(id, name, `Ticket store healthy · recent ${recent.length}:
${[...byState.entries()].map(([s, n]) => `${s}:${n}`).join(" · ") || "none yet"}`);
  } catch (e) {
    return fail(id, name, `Ticket gate error: ${(e as Error).message}`,
      "The confirmation gate is the ONLY path to a broker — if it errors, treat as critical. Inspect
queries/tickets.ts and the order_tickets table.");
  }
};

/* ---------- 9. intelligence (live ping) ---------- */

const checkIntelligence: CheckFn = async () => {
  const id = "intelligence";
  const name = "Intelligence (LLM)";
  const key = process.env.OPENAI_API_KEY;
  if (!key) {
    return warn(id, name, "OPENAI_API_KEY not set — chat runs on canned fallback answers only",
      "Set OPENAI_API_KEY (and optionally OPENAI_MODEL) in deployment to enable the reasoning
loop.");
  }
  try {
    const started = Date.now();
    const res = await fetch("https://api.openai.com/v1/chat/completions", {
      method: "POST",
      headers: { "Content-Type": "application/json", Authorization: `Bearer ${key}` },
      body: JSON.stringify({
        model: process.env.OPENAI_MODEL ?? "gpt-5.6",
        messages: [{ role: "user", content: "Reply with exactly: OK" }],
        max_tokens: 8,
      }),
    });
    const latency = Date.now() - started;
```

```
const body = (await res.text()).slice(0, 200);
      return fail(id, name, `OpenAI ping failed: HTTP ${res.status} — ${body}`,
        "Check the API key validity, billing/quota, and that OPENAI_MODEL names a model your account
can access. Intelligence falls back to canned answers while this fails.");
    }
    return ok(id, name, `Model ${process.env.OPENAI_MODEL ?? "gpt-5.6"} responding (${latency}ms)`);
  } catch (e) {
    return fail(id, name, `OpenAI ping error: ${(e as Error).message}`,
      "Network path to api.openai.com failed from this deployment — check egress/proxy. Intelligence
falls back to canned answers.");
  }
};

/* ---------- 10. memory ---------- */

const checkMemory: CheckFn = async () => {
  const id = "memory";
  const name = "Conversation memory";
  try {
    const db = getDb();
    const rows = await db.select({ id: chatMessages.id
}).from(chatMessages).orderBy(desc(chatMessages.id)).limit(1);
    return ok(id, name, `Memory table healthy (${rows.length > 0 ? "messages present" : "empty — no
chats yet, which is fine"})`);
  } catch (e) {
    return fail(id, name, `Memory error: ${(e as Error).message}`,
      "chat_messages table missing or unreadable — boot ensureSchema() should have created it; check
server logs and DATABASE_URL permissions.");
  }
};

/* ---------- 11. learning loop ---------- */

const checkLearning: CheckFn = async () => {
  const id = "learning";
  const name = "Learning loop";
  try {
    const db = getDb();
    const rows = await db.select({ id: tradeOutcomes.id
}).from(tradeOutcomes).orderBy(desc(tradeOutcomes.id)).limit(1);
    if (rows.length === 0) {
      return warn(id, name, "Learning table healthy but EMPTY — no outcomes logged yet, so the system
has nothing to learn from",
        "Run a backtest (Engine panel or ask Intelligence) — outcomes start flowing automatically
from there. Paper/live outcomes hook in as those paths close trades.");
    }
    return ok(id, name, "Learning table healthy with logged outcomes");
  } catch (e) {
    return fail(id, name, `Learning loop error: ${(e as Error).message}`,
      "trade_outcomes table missing or unreadable — boot ensureSchema() should have created it; check
server logs.");
  }
};

/* ---------- registry ---------- */

export const CHECKS: { fn: CheckFn }[] = [
  { fn: checkEnvironment },
  { fn: checkDatabase },
  { fn: checkGovernance },
  { fn: checkMarketData },
  { fn: checkEngineConfigs },
  { fn: checkSignals },
  { fn: checkAutonomous },
  { fn: checkTicketGate },
```

```
{ fn: checkMemory },
  { fn: checkLearning },
];

export async function runAllChecks(): Promise<CheckResult[]> {
  const results: CheckResult[] = [];
  for (const { fn } of CHECKS) {
    try {
      results.push(await fn());
    } catch (e) {
      results.push({ id: "unknown", name: fn.name, status: "FAIL", detail: `check crashed: ${(e as
Error).message}`, remediation: "Inspect the systemcheck module — a probe itself threw." });
    }
  }
  return results;
}
```

Appendix G — api/systemcheck/service.ts (new file, full source)

api/systemcheck/service.ts

Runner, AI diagnosis, notifications, ET scheduler. Create this file as-is.

```
import { desc, eq } from "drizzle-orm";
import { getDb } from "../queries/connection";
import { users, alerts, systemCheckReports } from "@db/schema";
import { etParts } from "../marketdata/indicators";
import { runAllChecks, type CheckResult } from "./checks";

/**
 * SYSTEM CHECK SERVICE — the platform's immune system.
 *
 * Runs the full component probe suite on a schedule (before market open,
 * after market close, ET, Mon–Fri) and on demand from the owner console.
 *
 *   1. PROBE — 11 real component checks (environment, database, governance,
 *      data feed, engine configs, signals self-test, autonomous monitors,
 *      ticket gate, intelligence, memory, learning).
 *   2. DIAGNOSE — when anything is degraded, the AI receives the failures
 *      and returns: what's broken, likely cause, exact fix steps, and one
 *      systemic improvement per failure — plus a short GROWTH section on
 *      making the platform better. On the daily post-close run it adds
 *      growth suggestions even when all checks pass.
 *   3. REPORT — every run persists to system_check_reports (owner console
 *      history) and pushes an in-app notification to all admin users via
 *      the alerts table (NotificationCenter polls it — no new UI plumbing).
 *
 * BOUNDARY: this module OBSERVES and ADVISES. The only automatic action it
 * ever takes is restarting the market-data refresh loop when it should be
 * running — everything else arrives as a recommendation for a human.
 */

export type SystemCheckTrigger = "PRE_OPEN" | "POST_CLOSE" | "MANUAL";

export interface StoredReport {
  id: number;
  triggerType: string;
  overall: string;
  okCount: number;
  warnCount: number;
  failCount: number;
  checks: CheckResult[];
  aiDiagnosis: string | null;
  createdAt: Date;
}

/* ---------- AI diagnosis ---------- */

async function diagnoseWithAi(results: CheckResult[], includeGrowth: boolean): Promise<string | null>
{
  const key = process.env.OPENAI_API_KEY;
  if (!key) return null;
  const issues = results.filter((r) => r.status !== "OK");
  if (issues.length === 0 && !includeGrowth) return null;

  const issueText = issues.length > 0
    ? issues.map((r) => `[${r.status}] ${r.name}: ${r.detail}${r.remediation ? ` (hint:
${r.remediation})` : ""}`).join("\n")
    : "All 11 component checks passed — no failures this run.";
```

```
const system = [
    "You are the chief reliability engineer of Requi Trading, an autonomous trading SaaS.",
    "Architecture: React/tRPC frontend; Hono server; Drizzle + MySQL; a deterministic strategy engine
(trigger library, setup state machine, backtester, simulator, scanner); a signed governance package
that clamps all risk config; an order-ticket confirmation gate (nothing reaches a broker without a
human CONFIRM); IBKR Client Portal market data; an LLM intelligence layer with tool access;
conversation memory; and a learning loop over logged trade outcomes.",
    "You receive system health-check results. Respond in plain text, concise and technical, with
these sections:",
    "BROKEN: for each FAIL/WARN — what is broken, the most likely cause, and the exact fix (env vars,
restart steps, file to inspect).",
    "PREVENT: one systemic improvement per failure class so it cannot recur silently.",
    "GROWTH: 2-3 concrete suggestions to make the whole platform better this week (reliability,
observability, or trading quality).",
    "Never suggest loosening risk limits, bypassing governance, or auto-executing without
confirmation. If you suggest it, it must keep those invariants.",
  ].join("\n");

  try {
    const res = await fetch("https://api.openai.com/v1/chat/completions", {
      method: "POST",
      headers: { "Content-Type": "application/json", Authorization: `Bearer ${key}` },
      body: JSON.stringify({
        model: process.env.OPENAI_MODEL ?? "gpt-5.6",
        messages: [
          { role: "system", content: system },
          { role: "user", content: `Health check results (${new
Date().toISOString()}):\n${issueText}` },
        ],
        max_tokens: 1400,
      }),
    });
    if (!res.ok) return null;
    const data = (await res.json()) as { choices?: { message?: { content?: string } }[] };
    return data.choices?.[0]?.message?.content?.trim() ?? null;
  } catch {
    return null;
  }
}

/* ---------- notifications (in-app, via the alerts table) ---------- */

async function notifyAdmins(overall: string, results: CheckResult[], diagnosis: string | null,
trigger: SystemCheckTrigger) {
  const db = getDb();
  const admins = await db.select({ id: users.id }).from(users).where(eq(users.role, "admin"));
  if (admins.length === 0) return;

  const issues = results.filter((r) => r.status !== "OK");
  const fails = issues.filter((r) => r.status === "FAIL");
  const triggerLabel = trigger === "PRE_OPEN" ? "pre-open" : trigger === "POST_CLOSE" ? "post-close"
: "manual";

  // Noise policy: WARN/FAIL always notify; a clean run only notifies on the
  // daily post-close (a once-a-day "all clear"), never on pre-open/manual spam.
  const shouldNotify = overall !== "OK" || trigger === "POST_CLOSE";
  if (!shouldNotify) return;

  const title =
    overall === "FAIL"
      ? `System Check FAILED — ${fails.length} component(s) down`
      : overall === "WARN"
        ? `System Check: ${issues.length} warning(s) detected`
        : "System Check: all clear";
```

```
`${triggerLabel} run · ${results.filter((r) => r.status === "OK").length} OK · ${issues.filter((r) =>
r.status === "WARN").length} WARN · ${fails.length} FAIL`,
    ...issues.slice(0, 5).map((r) => `${r.status} — ${r.name}: ${r.detail}`),
  ];
  if (diagnosis) bodyLines.push("", "AI diagnosis:", diagnosis.slice(0, 900));
  else if (overall !== "OK") bodyLines.push("", "(AI diagnosis unavailable — OPENAI_API_KEY missing
or model error; see owner System Check page for check details.)");

  for (const a of admins) {
    await db.insert(alerts).values({
      userId: a.id,
      type: "SYSTEM_STATE",
      priority: overall === "FAIL" ? "CRITICAL" : overall === "WARN" ? "HIGH" : "LOW",
      title,
      body: bodyLines.join("\n").slice(0, 2000),
      state: overall === "OK" ? "WATCHING" : "ACTION_REQUIRED",
    });
  }
}

/* ---------- runner ---------- */

let running = false;

export async function runSystemCheck(trigger: SystemCheckTrigger): Promise<StoredReport> {
  if (running) {
    const latest = await getLatestReport();
    if (latest) return latest;
  }
  running = true;
  try {
    const results = await runAllChecks();
    const failCount = results.filter((r) => r.status === "FAIL").length;
    const warnCount = results.filter((r) => r.status === "WARN").length;
    const okCount = results.length - failCount - warnCount;
    const overall = failCount > 0 ? "FAIL" : warnCount > 0 ? "WARN" : "OK";

    // AI diagnosis: on issues always; on the daily post-close even when clean (growth suggestions).
    const diagnosis = await diagnoseWithAi(results, trigger === "POST_CLOSE");

    const db = getDb();
    const [inserted] = await db
      .insert(systemCheckReports)
      .values({
        triggerType: trigger,
        overall,
        okCount,
        warnCount,
        failCount,
        checksJson: JSON.stringify(results),
        aiDiagnosis: diagnosis,
      })
      .$returningId();

    await notifyAdmins(overall, results, diagnosis, trigger).catch(() => undefined);

    return {
      id: inserted?.id ?? 0,
      triggerType: trigger,
      overall,
      okCount,
      warnCount,
      failCount,
      checks: results,
      aiDiagnosis: diagnosis,
```

```
};
  } finally {
    running = false;
  }
}

/* ---------- history ---------- */

function toStored(row: typeof systemCheckReports.$inferSelect): StoredReport {
  let checks: CheckResult[] = [];
  try {
    checks = JSON.parse(row.checksJson) as CheckResult[];
  } catch {
    checks = [];
  }
  return {
    id: Number(row.id),
    triggerType: row.triggerType,
    overall: row.overall,
    okCount: row.okCount,
    warnCount: row.warnCount,
    failCount: row.failCount,
    checks,
    aiDiagnosis: row.aiDiagnosis ?? null,
    createdAt: row.createdAt,
  };
}

export async function getLatestReport(): Promise<StoredReport | null> {
  const db = getDb();
  const rows = await
db.select().from(systemCheckReports).orderBy(desc(systemCheckReports.id)).limit(1);
  return rows.length > 0 ? toStored(rows[0]) : null;
}

export async function listReports(limit = 20): Promise<StoredReport[]> {
  const db = getDb();
  const rows = await
db.select().from(systemCheckReports).orderBy(desc(systemCheckReports.id)).limit(Math.min(limit, 50));
  return rows.map(toStored);
}

/* ---------- scheduler: pre-open 09:00 ET, post-close 16:15 ET, Mon–Fri ---------- */

const PRE_OPEN_MIN = 9 * 60; // 09:00 ET, window 09:00–09:25
const POST_CLOSE_MIN = 16 * 60 + 15; // 16:15 ET, window 16:15–16:40
const WINDOW = 25;

let schedulerTimer: NodeJS.Timeout | null = null;
const ranSlots = new Set<string>();

function etWeekday(now: number): number {
  return new Date(new Date(now).toLocaleString("en-US", { timeZone: "America/New_York" })).getDay();
}

async function schedulerTick() {
  const now = Date.now();
  const dow = etWeekday(now);
  if (dow === 0 || dow === 6) return;
  const { day, minutes } = etParts(now);

  if (minutes >= PRE_OPEN_MIN && minutes < PRE_OPEN_MIN + WINDOW && !ranSlots.has(`${day}:PRE_OPEN`))
{
    ranSlots.add(`${day}:PRE_OPEN`);
    await runSystemCheck("PRE_OPEN").catch(() => ranSlots.delete(`${day}:PRE_OPEN`));
```

```
if (minutes >= POST_CLOSE_MIN && minutes < POST_CLOSE_MIN + WINDOW &&
!ranSlots.has(`${day}:POST_CLOSE`)) {
    ranSlots.add(`${day}:POST_CLOSE`);
    await runSystemCheck("POST_CLOSE").catch(() => ranSlots.delete(`${day}:POST_CLOSE`));
  }
  // bound the latch set (keep today + yesterday only)
  if (ranSlots.size > 6) {
    const keep = [...ranSlots].slice(-4);
    ranSlots.clear();
    for (const k of keep) ranSlots.add(k);
  }
}

export function ensureSystemCheckLoop(): void {
  if (schedulerTimer) return;
  schedulerTimer = setInterval(() => {
    schedulerTick().catch(() => undefined);
  }, 60_000);
  schedulerTimer.unref();
}

export function systemCheckLoopStatus(): { running: boolean; ranToday: string[] } {
  return { running: schedulerTimer !== null, ranToday: [...ranSlots] };
}
```

Appendix H — api/systemcheck-router.ts (new file, full source)

api/systemcheck-router.ts

Owner-only routes. Create this file as-is.

```
import { createRouter, adminQuery } from "./middleware";
import { getLatestReport, listReports, runSystemCheck, systemCheckLoopStatus } from
"./systemcheck/service";

/**
 * SYSTEM CHECK API — owner-only (admin role). The platform's health and
 * self-diagnosis surface: latest report, history, manual run, scheduler
 * status. Scheduled runs (pre-open / post-close ET) happen server-side;
 * these routes expose them to the owner console.
 */
export const systemCheckRouter = createRouter({
  latest: adminQuery.query(async () => ({
    report: await getLatestReport(),
    loop: systemCheckLoopStatus(),
  })),

  history: adminQuery.query(async () => listReports(20)),

  runNow: adminQuery.mutation(async () => runSystemCheck("MANUAL")),
});
```

Appendix I — api/router.ts + api/boot.ts (edits)

I1 — api/router.ts

Import and register the systemCheck router.

```
import { marketDataRouter } from "./marketdata-router";
import { engineRouter } from "./engine-router";
import { systemCheckRouter } from "./systemcheck-router";

// ...inside createRouter({...}):
  marketData: marketDataRouter,
  engine: engineRouter,
  systemCheck: systemCheckRouter,
});
```

I2 — api/boot.ts

Import and start the scheduler at the top of the production block.

```
import { ensureSchema } from "./lib/ensure-schema";
import { ensureGovernanceReady } from "./governance/runtime";
import { ensureSystemCheckLoop } from "./systemcheck/service";

if (env.isProduction) {
  await ensureSchema(); // fresh databases work on first boot — live stream, alerts, limits
  await ensureGovernanceReady(); // registry sync + signed genesis governance package if none active
  ensureSystemCheckLoop(); // pre-open (09:00 ET) + post-close (16:15 ET) health runs, Mon–Fri
```

Appendix J — db/schema.ts (append these table definitions)

db/schema.ts — additions

Append after the existing tables. bigint/text/int/decimal/timestamp/mysqlEnum/serial/varchar are already imported in this file.

```
/* ---------- Intelligence: conversation memory ---------- */

export const chatMessages = mysqlTable("chat_messages", {
  id: serial("id").primaryKey(),
  userId: bigint("userId", { mode: "number", unsigned: true }).notNull(),
  role: varchar("role", { length: 16 }).notNull(), // 'user' | 'assistant'
  content: text("content").notNull(),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
});
export type ChatMessageRow = typeof chatMessages.$inferSelect;

/* ---------- Learning loop: logged trade outcomes ---------- */

export const tradeOutcomes = mysqlTable("trade_outcomes", {
  id: serial("id").primaryKey(),
  userId: bigint("userId", { mode: "number", unsigned: true }).notNull(),
  symbol: varchar("symbol", { length: 16 }).notNull(),
  strategyId: varchar("strategyId", { length: 64 }).notNull(),
  variantId: varchar("variantId", { length: 64 }),
  source: mysqlEnum("source", ["BACKTEST", "SIMULATION", "PAPER", "LIVE"]).notNull(),
  entry: decimal("entry", { precision: 12, scale: 4 }),
  exitPrice: decimal("exitPrice", { precision: 12, scale: 4 }),
  qty: int("qty"),
  pnl: decimal("pnl", { precision: 12, scale: 2 }),
  rMultiple: decimal("rMultiple", { precision: 8, scale: 3 }),
  exitReason: varchar("exitReason", { length: 32 }),
  lesson: varchar("lesson", { length: 255 }),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
});
export type TradeOutcome = typeof tradeOutcomes.$inferSelect;

/* ---------- System check module: health reports (owner-facing) ---------- */

export const systemCheckReports = mysqlTable("system_check_reports", {
  id: serial("id").primaryKey(),
  triggerType: varchar("triggerType", { length: 24 }).notNull(), // 'PRE_OPEN' | 'POST_CLOSE' |
'MANUAL'
  overall: varchar("overall", { length: 8 }).notNull(), // 'OK' | 'WARN' | 'FAIL'
  okCount: int("okCount").notNull().default(0),
  warnCount: int("warnCount").notNull().default(0),
  failCount: int("failCount").notNull().default(0),
  checksJson: text("checksJson").notNull(),
  aiDiagnosis: text("aiDiagnosis"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
});
export type SystemCheckReport = typeof systemCheckReports.$inferSelect;
```

Appendix K — api/lib/ensure-schema.ts (additions)

K1 — STATEMENTS array

Add the three CREATE TABLE statements at the end of the STATEMENTS array (before the closing bracket). All idempotent — safe on every boot.

```
  `CREATE TABLE IF NOT EXISTS chat_messages (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
    userId BIGINT UNSIGNED NOT NULL,
    role VARCHAR(16) NOT NULL,
    content TEXT NOT NULL,
    createdAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_chat_user (userId, createdAt)
  )`,
  `CREATE TABLE IF NOT EXISTS trade_outcomes (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
    userId BIGINT UNSIGNED NOT NULL,
    symbol VARCHAR(16) NOT NULL,
    strategyId VARCHAR(64) NOT NULL,
    variantId VARCHAR(64),
    source ENUM('BACKTEST','SIMULATION','PAPER','LIVE') NOT NULL,
    entry DECIMAL(12,4),
    exitPrice DECIMAL(12,4),
    qty INT,
    pnl DECIMAL(12,2),
    rMultiple DECIMAL(8,3),
    exitReason VARCHAR(32),
    lesson VARCHAR(255),
    createdAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_outcomes_user (userId, strategyId)
  )`,
  `CREATE TABLE IF NOT EXISTS system_check_reports (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
    triggerType VARCHAR(24) NOT NULL,
    overall VARCHAR(8) NOT NULL,
    okCount INT NOT NULL DEFAULT 0,
    warnCount INT NOT NULL DEFAULT 0,
    failCount INT NOT NULL DEFAULT 0,
    checksJson TEXT NOT NULL,
    aiDiagnosis TEXT,
    createdAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
  )`,
];
```

K2 — USER_ID_TABLES

Add chat_messages and trade_outcomes so duplicate-user merges repoint them. (system_check_reports is system-level, not user-scoped.)

```
const USER_ID_TABLES = ["strategies", "broker_accounts", "alerts", "run_events", "run_sessions",
"ai_limits", "support_tickets", "chat_messages", "trade_outcomes"];
```

Appendix L — src/pages/app/owner/SystemCheck.tsx (new file, full source)

src/pages/app/owner/SystemCheck.tsx

Owner console page. Create this file as-is.

```
import {
  Activity, CheckCircle2, XCircle, AlertTriangle, PlayCircle, RefreshCw,
  HeartPulse, BrainCircuit, Clock,
} from 'lucide-react';
import { trpc } from '@/providers/trpc';
import { Badge, PageHeader, StatCard, TableShell, th, thCls, trCls } from './shared';

interface CheckResult {
  id: string;
  name: string;
  status: 'OK' | 'WARN' | 'FAIL';
  detail: string;
  remediation?: string;
}
interface StoredReport {
  id: number;
  triggerType: string;
  overall: string;
  okCount: number;
  warnCount: number;
  failCount: number;
  checks: CheckResult[];
  aiDiagnosis: string | null;
  createdAt: string | Date;
}

function StatusIcon({ status }: { status: string }) {
  if (status === 'OK') return <CheckCircle2 className="h-4 w-4 text-teal-600" />;
  if (status === 'FAIL') return <XCircle className="h-4 w-4 text-red-600" />;
  return <AlertTriangle className="h-4 w-4 text-amber-500" />;
}
function statusTone(s: string): 'teal' | 'amber' | 'red' {
  return s === 'OK' ? 'teal' : s === 'FAIL' ? 'red' : 'amber';
}
function triggerLabel(t: string) {
  return t === 'PRE_OPEN' ? 'Pre-open' : t === 'POST_CLOSE' ? 'Post-close' : 'Manual';
}

export default function SystemCheck() {
  const utils = trpc.useUtils();
  const { data, refetch, isFetching } = trpc.systemCheck.latest.useQuery(undefined, {
refetchInterval: 30000, retry: false });
  const { data: history } = trpc.systemCheck.history.useQuery(undefined, { retry: false });
  const runNow = trpc.systemCheck.runNow.useMutation({
    onSuccess: () => {
      utils.systemCheck.latest.invalidate();
      utils.systemCheck.history.invalidate();
    },
  });

  const report = data?.report as StoredReport | null | undefined;
  const loop = data?.loop as { running: boolean; ranToday: string[] } | undefined;

  return (
    <div className="space-y-6">
      <PageHeader
```

```
title="System Check"
        description="Automated health probes across every major component — data feed, engine,
signals, autonomous monitors, ticket gate, intelligence, memory, and the learning loop. Runs
automatically before market open (09:00 ET) and after close (16:15 ET), Mon–Fri. Failures are
diagnosed by AI and pushed to your notifications."
        actions={
          <button
            onClick={() => runNow.mutate()}
            disabled={runNow.isPending}
            className="inline-flex items-center gap-2 rounded-xl bg-royal-600 px-4 py-2 text-sm font-
semibold text-white transition hover:bg-royal-500 disabled:opacity-50"
          >
            {runNow.isPending ? <RefreshCw className="h-4 w-4 animate-spin" /> : <PlayCircle
className="h-4 w-4" />}
            {runNow.isPending ? 'Running 11 checks…' : 'Run now'}
          </button>
        }
      />

      {/* scheduler + overall */}
      <div className="grid gap-4 sm:grid-cols-3">
        <StatCard
          icon={HeartPulse}
          label="Latest overall"
          value={report ? report.overall : '—'}
          sub={report ? `${triggerLabel(report.triggerType)} run · ${new
Date(report.createdAt).toLocaleString()}` : 'No runs yet — click Run now'}
        />
        <StatCard
          icon={Activity}
          label="Components"
          value={report ? `${report.okCount} / ${report.okCount + report.warnCount +
report.failCount} OK` : '—'}
          sub={report ? `${report.warnCount} warning(s) · ${report.failCount} failure(s)` :
undefined}
        />
        <StatCard
          icon={Clock}
          label="Scheduler"
          value={loop?.running ? 'Active' : 'Off'}
          sub={loop?.ranToday?.length ? `Today: ${loop.ranToday.map((s) => s.split(':')[1]).join(',
')}` : 'Pre-open 09:00 ET · Post-close 16:15 ET'}
        />
      </div>

      {/* component table */}
      <TableShell
        title="Component checks"
        aside={
          <button onClick={() => refetch()} className="inline-flex items-center gap-1.5 text-xs font-
semibold text-slate-500 hover:text-slate-700">
            <RefreshCw className={`h-3.5 w-3.5 ${isFetching ? 'animate-spin' : ''}`} /> Refresh
          </button>
        }
      >
        {!report && <p className="px-6 py-10 text-center text-sm text-slate-500">No system check has
run yet. Use “Run now” for the first report — scheduled runs happen automatically around market
hours.</p>}
        {report && (
          <table className="w-full text-sm">
            <thead>
              <tr className={thCls}>
                <th className={th}>Component</th>
                <th className={th}>Status</th>
                <th className={th}>Detail</th>
```

```
</tr>
            </thead>
            <tbody>
              {report.checks.map((c) => (
                <tr key={c.id} className={trCls}>
                  <td className="px-6 py-3 font-medium text-slate-800">
                    <span className="inline-flex items-center gap-2"><StatusIcon status={c.status} />
{c.name}</span>
                  </td>
                  <td className="px-6 py-3"><Badge tone={statusTone(c.status)}>{c.status}</Badge>
</td>
                  <td className="max-w-md px-6 py-3 text-slate-600">{c.detail}</td>
                  <td className="max-w-xs px-6 py-3 text-xs text-slate-500">{c.remediation ?? '—'}
</td>
                </tr>
              ))}
            </tbody>
          </table>
        )}
      </TableShell>

      {/* AI diagnosis */}
      {report?.aiDiagnosis && (
        <div className="glass rounded-2xl p-6">
          <div className="mb-3 flex items-center gap-2">
            <BrainCircuit className="h-4 w-4 text-violet-600" />
            <h3 className="font-display text-base font-bold text-slate-900">AI diagnosis &
improvement plan</h3>
          </div>
          <pre className="whitespace-pre-wrap font-sans text-sm leading-relaxed text-slate-700">
{report.aiDiagnosis}</pre>
        </div>
      )}

      {/* history */}
      <TableShell title="Run history">
        {!history || history.length === 0 ? (
          <p className="px-6 py-10 text-center text-sm text-slate-500">No history yet.</p>
        ) : (
          <table className="w-full text-sm">
            <thead>
              <tr className={thCls}>
                <th className={th}>When</th>
                <th className={th}>Trigger</th>
                <th className={th}>Overall</th>
                <th className={th}>OK / Warn / Fail</th>
                <th className={th}>AI diagnosis</th>
              </tr>
            </thead>
            <tbody>
              {(history as StoredReport[]).map((r) => (
                <tr key={r.id} className={trCls}>
                  <td className="px-6 py-3 text-slate-600">{new Date(r.createdAt).toLocaleString()}
</td>
                  <td className="px-6 py-3 text-slate-600">{triggerLabel(r.triggerType)}</td>
                  <td className="px-6 py-3"><Badge tone={statusTone(r.overall)}>{r.overall}</Badge>
</td>
                  <td className="px-6 py-3 text-slate-600">{r.okCount} / {r.warnCount} /
{r.failCount}</td>
                  <td className="px-6 py-3 text-slate-500">{r.aiDiagnosis ? 'Yes' : '—'}</td>
                </tr>
              ))}
            </tbody>
          </table>
        )}
```

```
</div>
  );
}
```

Appendix M — src/App.tsx + src/pages/app/AppLayout.tsx (edits)

M1 — src/App.tsx

Import the page and add the route alongside the other owner routes.

```
import OwnerGovernance from './pages/app/owner/Governance';
import OwnerSystemCheck from './pages/app/owner/SystemCheck';

// ...inside the owner routes:
        <Route path="owner/governance" element={<OwnerGovernance />} />
        <Route path="owner/system-check" element={<OwnerSystemCheck />} />
        <Route path="owner/support" element={<OwnerSupport />} />
```

M2 — src/pages/app/AppLayout.tsx

Add HeartPulse to the lucide imports and the nav item to ownerNav.

```
  ChevronDown, Menu, X, Zap, LayoutDashboard, Users, CreditCard, Plug2, LifeBuoy, ShieldCheck,
HeartPulse,
} from 'lucide-react';

const ownerNav = [
  { to: '/app/owner', label: 'Owner Overview', icon: LayoutDashboard, end: true },
  { to: '/app/owner/users', label: 'Users', icon: Users },
  { to: '/app/owner/billing', label: 'Billing', icon: CreditCard },
  { to: '/app/owner/connectors', label: 'MCP Connectors', icon: Plug2 },
  { to: '/app/owner/governance', label: 'Governance', icon: ShieldCheck },
  { to: '/app/owner/system-check', label: 'System Check', icon: HeartPulse },
  { to: '/app/owner/support', label: 'Support', icon: LifeBuoy },
];
```